Online Resource 1: Supplementary Figures and PLENA Implementation Notes

Article Title: High-Performance C++ Reimplementation and Optimization of the Gibbs' Model for Analyzing Structural Complexity in Urban Drainage Networks

Journal: Natural Hazards

Authors: Changmin Park, Minsoo Seok, Yongwon Seo

Affiliation: Department of Civil Engineering, Yeungnam University, Gyeongsan, Republic of Korea

Corresponding Author: Yongwon Seo (yongwon.seo@yu.ac.kr)

1. Scope of this Online Resource

Purpose. This Online Resource provides supplementary computational notes and figure explanations for PLENA, the C++ implementation used for Gibbs' Model-based drainage-network generation, width-function computation, Nash-Sutcliffe Efficiency comparison, and representative beta estimation. It is written as supporting material for the PLENA manuscript and does not present FMT as a completed or validated method.

Manuscript connection. The manuscript describes PLENA as a C++17 framework that preserves the Gibbs' Model workflow while reorganizing the computation for repeated stochastic simulations and city-scale drainage-catchment analysis. The same manuscript identifies the PLENA source-code repository as the location for implementation details, build instructions, input direction coding, output-file descriptions, license information, and implementation provenance.

2. Implementation Components Documented by the Supplementary Figures

Contiguous matrix storage. PLENA defines a Matrix structure backed by one contiguous integer vector data array. The row-major accessor maps a two-dimensional cell coordinate to a one-dimensional offset. This implementation detail is relevant because repeated neighbor access occurs during flow-direction updates, reachability checks, travel-time computation, and width-function construction. Figure S1 illustrates the cache-locality rationale for using contiguous storage rather than nested containers.

Candidate screening and reachability. At each stochastic update, PLENA proposes a local flow-direction change and rejects infeasible candidates before the global reachability check. The verified code rejects repeated directions, boundary violations, movement to inactive cells, and immediate opposite-direction conflicts with adjacent

cells. A breadth-first reachability search from the outlet is then used to verify that valid cells remain connected to the outlet.

Scale-consistent iteration control. In the version checked for this Online Resource, the Gibbs update loop uses a maximum iteration count of 10 x n x m, where n and m are the grid dimensions. This makes the number of stochastic update attempts proportional to catchment-grid size rather than fixed independently of the number of cells.

Parallel batch execution. PLENA's beta-class and replicate-run computations are independent tasks. The verified implementation resolves the thread count from hardware_concurrency(), applies a user-specified thread cap when provided, and uses a C++ thread worker pool with atomic task indexing. Figure S2 documents this parallel execution with console output and CPU-utilization evidence.

3. Runtime Comparison and Output Files

Runtime comparison. The manuscript reports that PLENA reproduced the fundamental behavior of the MATLAB-based Gibbs' Model framework while substantially reducing computation time. Depending on grid size and computing environment, the reported runtime acceleration is approximately 1,380 to 38,700 times faster than the conventional MATLAB implementation. Figure S3 presents the corresponding generation-time comparison across network sizes.

Output files. The verified source code writes text and CSV outputs for the final direction matrix, travel-time matrix, flow-accumulation matrix, q(t), width-function distributions, candidate beta summaries, and NSE reports. These outputs support the manuscript's width-function-based beta estimation workflow.

Source basis. This Online Resource was prepared from the local instruction file, the manuscript file NH_PLENA_manuscript_last5.docx, the checked PLENA_FINAL main.cpp source file, and the three supplied figure files Figure S1.tif, Figure S2.tif, and Figure S3.tif.

Figures

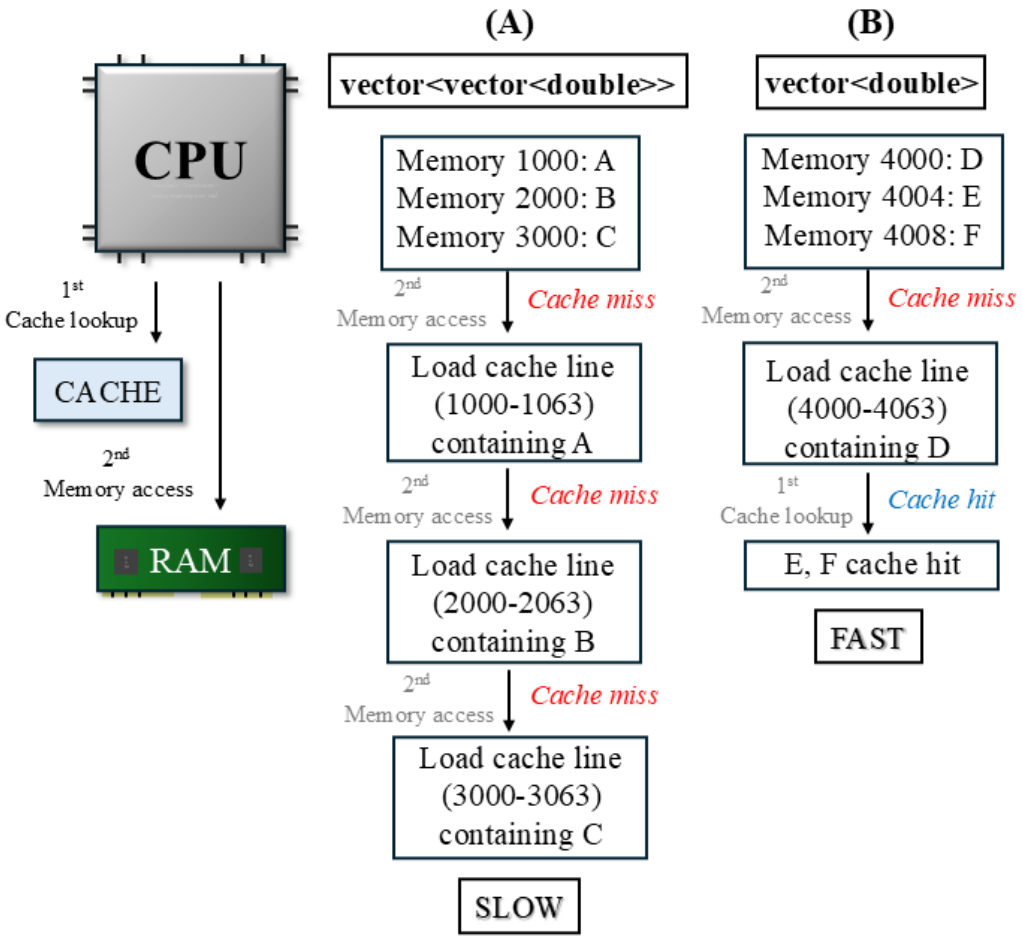

Figure S1. Schematic illustration of cache hits and cache misses for different memory layouts. The left-hand workflow shows repeated cache misses when grid-like data are represented as separated nested containers. The right-hand workflow shows how contiguous memory can improve cache locality because adjacent values are loaded into the same cache line. This figure supports the PLENA implementation choice of contiguous matrix storage.

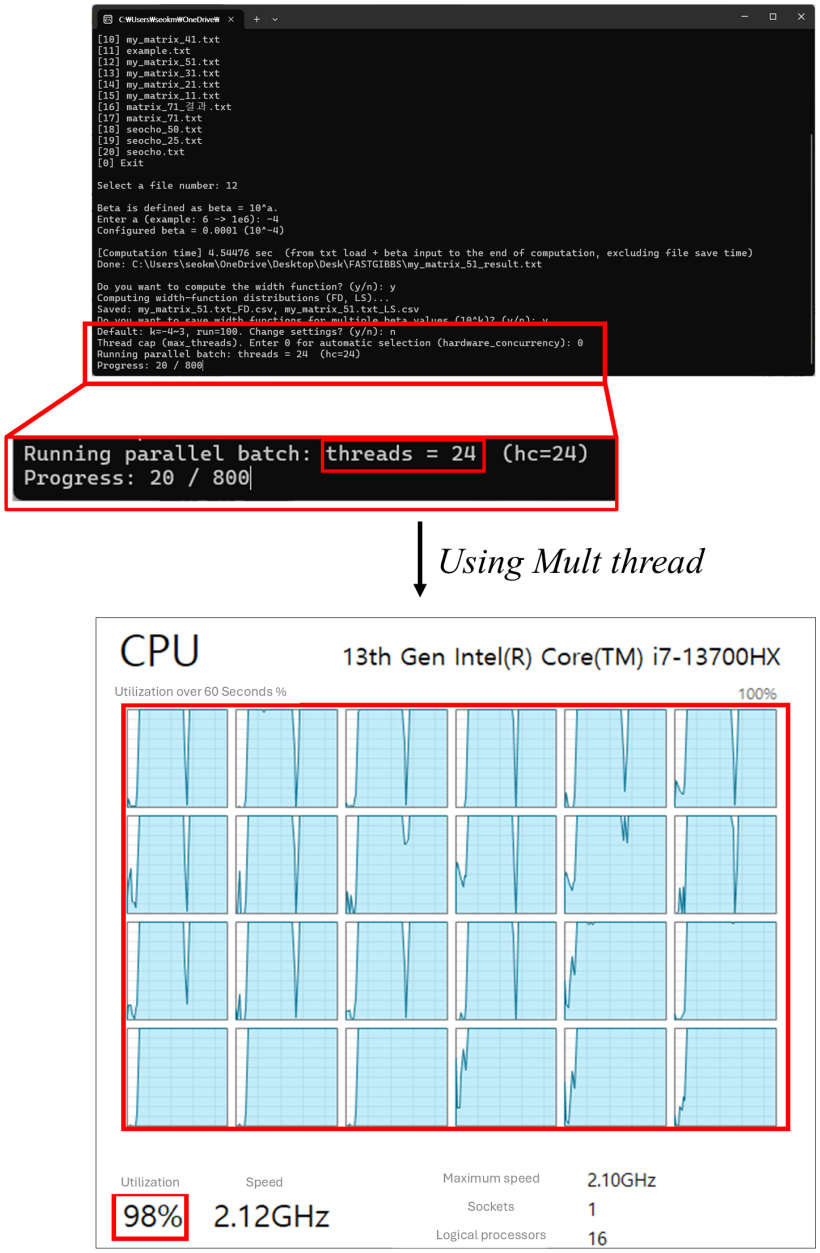

Figure S2. Console output and CPU utilization during PLENA execution with multi-threading. The highlighted console message reports automatic thread selection and batch progress, including the use of 24 threads in the captured run. The CPU-utilization panel shows high processor utilization during parallel width-function estimation.

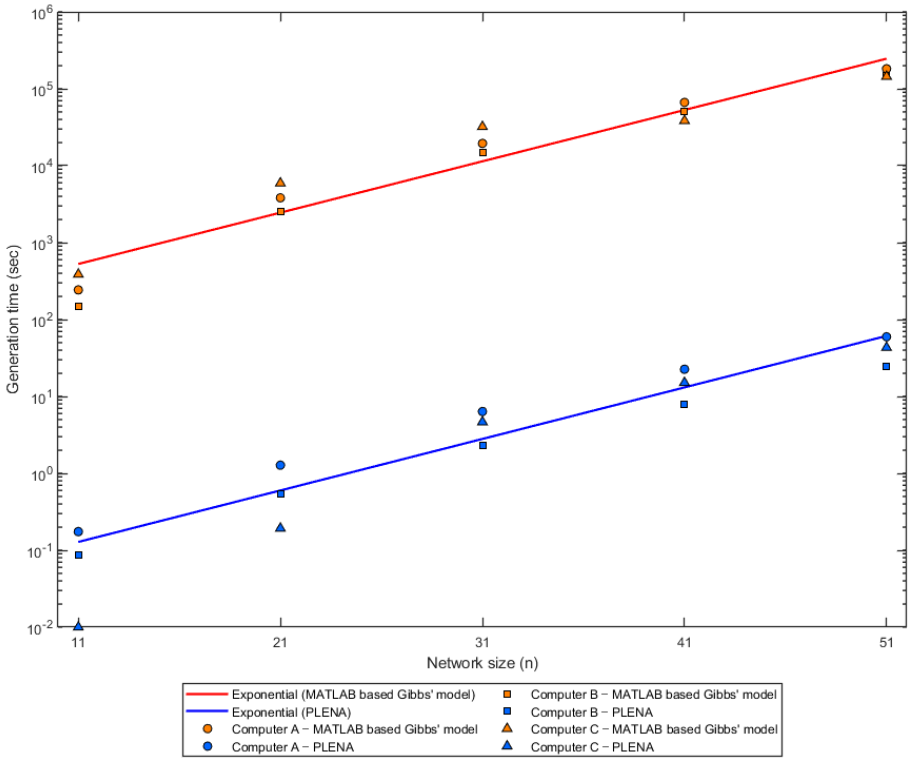

Figure S3. Generation-time comparison between the MATLAB-based Gibbs' Model implementation and PLENA across network sizes. The y-axis is logarithmic, and the PLENA series remains far below the MATLAB-based series across the plotted network sizes. This figure documents computational acceleration for single-network regeneration conditions and supports the manuscript's statement that PLENA enables repeated width-function-based analysis at larger scale.